

Referencia de módulos

API completa: parámetros, tipos, retornos y ejemplos — derivada del código fuente

Descripción completa de la superficie pública de cada módulo del sistema. Toda la información está derivada directamente del código fuente.

ingesta.py — Carga y limpieza de datos

leer_csv(ruta_archivo)

Lee un CSV y retorna una lista de registros.

Parámetro	Tipo	Descripción
<code>ruta_archivo</code>	<code>str</code>	Ruta al archivo CSV (relativa o absoluta)

Retorna: `list[dict]` — cada dict tiene claves `id` (`str`), `texto` (`str`), `fecha` (`str`) y `usuario` (`str`; cadena vacía si la columna no existe en el CSV).

Excepciones: `FileNotFoundError` — el archivo no existe; `ValueError` — el CSV no tiene las columnas `id`, `texto`, `fecha`.

Columnas: `id`, `texto` y `fecha` son obligatorias. `usuario` es opcional y participa en alertas, persistencia y reportes cuando está presente. Otras columnas adicionales se ignoran.

Codificación: intenta leer en UTF-8 primero. Si falla con `UnicodeDecodeError`, reintentará en latin-1. En ambos casos valida las columnas requeridas antes de procesar filas.

Filas ignoradas: filas donde todos los campos son cadenas vacías.

limpiar_texto(texto)

Normaliza texto para el análisis de sentimiento.

Retorna: `str` — texto en minúsculas, sin caracteres especiales (excepto letras acentuadas del español, ñ, dígitos y espacios), con espacios múltiples colapsados. Si la entrada no es `str`, se convierte con `str()`.

```
from ingest import limpiar_texto
limpiar_texto("¡Terrible! Esto es UN FRAUDE.") # → "terrible esto es un fraude"
limpiar_texto("Muy bien :) !!!!")           # → "muy bien"
```

detectar_duplicados(datos)

Separa registros únicos de duplicados basándose en el texto limpio.

Retorna: `tuple[list[dict], list[dict]]` — `datos_unicos` (primer registro de cada texto único, con clave `texto_limpio` añadida) y `duplicados` (registros que repiten un texto ya visto, cualquiera sea su id o fecha).

procesar_datos(ruta_archivo)

Pipeline completo: lectura → limpieza → deduplicación → filtro de ruido.

El filtro de ruido descarta los registros cuyo texto limpio tiene menos de `LONGITUD_MINIMA = 3` caracteres (símbolos sueltos, restos de enlaces, etc.).

Retorna: `tuple[list[dict], dict]` — `datos_procesados` (registros únicos válidos con `texto_limpio`) y `estadisticas` con claves:

- `total_original` (`int`): filas leídas del CSV
- `total_unicos` (`int`): filas válidas después de deduplicar y filtrar ruido
- `total_duplicados` (`int`): filas descartadas por duplicación
- `duplicados_ids` (`list[str]`): IDs de los duplicados descartados
- `total_ruido` (`int`): filas descartadas por el filtro de ruido

Excepciones: propaga `FileNotFoundError` y `ValueError` de `leer_csv`.

clasificador.py — Análisis de sentimiento

AnalizadorBase

Superclase de los analizadores del sistema (evidencia de **herencia y polimorfismo** de la Unidad 2). Define el atributo `nombre`, el método `describir()` y una versión genérica de `clasificar()` que retorna siempre "neutro"; las subclases sobrescriben este método con su propia lógica.

AnalizadorSentimiento(AnalizadorBase)

Analiza el sentimiento de textos en español mediante un léxico ponderado de **256 entradas**, que cubre vocabulario formal, coloquialismos y jerga peruana, además de intensificadores. No utiliza modelos preentrenados de IA: la clasificación es puramente léxica con la biblioteca estándar de Python.

Constructor

```
AnalizadorSentimiento(umbral_muy_positivo=2.0, umbral_muy_negativo=-2.0)
```

Parámetro	Tipo	Default	Descripción
umbral_muy_positivo	float	2.0	Puntuación normalizada mínima para "muy positivo"
umbral_muy_negativo	float	-2.0	Puntuación normalizada máxima para "muy negativo"

Constantes de categoría

```
AnalizadorSentimiento.MUY_POSITIVO # "muy positivo"
AnalizadorSentimiento.POSITIVO     # "positivo"
AnalizadorSentimiento.NEUTRO       # "neutro"
AnalizadorSentimiento.NEGATIVO     # "negativo"
AnalizadorSentimiento.MUY_NEGATIVO # "muy negativo"
```

Escala del léxico

Peso	Significado	Ejemplos
+3	Muy positivo	maravilloso, excelente, espectacular
+2	Positivo	bueno, mejora, recomiendo
+1	Levemente positivo	aceptable, normal, razonable
-1	Levemente negativo	preocupante, retraso, deficiente
-2	Negativo	malo, denuncia, negligencia, vergüenza
-3	Muy negativo	fraude, escándalo, corrupto, jerga ofensiva
+0.3 a +1.0	Intensificador	muy (0.5), absolutamente (1.0), siempre (0.3)

Los intensificadores suman su peso propio a la puntuación bruta (por eso "absolutamente terrible" puntúa peor que "terrible" a igual longitud).

Negadores

Las palabras `no`, `nunca`, `jamás / jamas`, `ni`, `tampoco` y `sin` invierten el signo del peso de la palabra que les sigue inmediatamente; la pareja negador + palabra consume ambos tokens.

```
"no bueno"    → bueno (+2) negado → -2
"nunca malo"  → malo (-2) negado → +2
```

Si el negador es la última palabra de la oración, no suma ningún peso.

clasificar(texto_limpio)

Clasifica el sentimiento de un texto ya normalizado. Sobreescribe el método genérico de `AnalizadorBase` (polimorfismo).

Retorna: dict con claves `categoria` (str), `puntuacion` (float, redondeada a 4 decimales), `puntuacion_raw` (float) y `palabras_clave` (list[str]: palabras con peso negativo presentes en el texto).

Fórmula de puntuación:

```
puntuacion = round((puntuacion_raw / max(n_palabras, 1)) * 10, 4)
```

Tabla de categorías

Puntuación normalizada	Categoría
≥ 2.0	muy positivo
> 0 y < 2.0	positivo
= 0	neutro
≥ -2.0 y < 0	negativo
< -2.0	muy negativo

```
from clasificador import AnalizadorSentimiento
an = AnalizadorSentimiento()
an.clasificar("absolutamente terrible un completo desastre")
# → {
#   'categoria': 'muy negativo',
#   'puntuacion': -10.0,
#   'puntuacion_raw': -5.0,
```

```
# 'palabras_clave': ['terrible', 'desastre']
# }
# absolutamente (+1.0) + terrible (-3) + desastre (-3) = -5.0 raw
# normalizado: (-5.0 / 5 palabras) * 10 = -10.0
```

analizar_lote(datos)

Analiza una lista de comentarios en lote usando `map()`. Cada registro original recibe las claves de análisis: `categoria`, `puntuacion`, `puntuacion_raw`, `palabras_clave`.

filtrar_negativos(datos_analizados, umbral=-1.0)

Filtra comentarios cuya puntuación está por debajo del umbral dado (usa `filter()` con `lambda`). **Retorna:** `list(dict)`.

obtener_estadisticas(datos_analizados)

Retorna: dict — mapa `categoria → {'count': int, 'porcentaje': float}` para las cinco categorías, en orden de muy positivo a muy negativo.

```
estadisticas = an.obtener_estadisticas(datos_analizados)
estadisticas["negativo"] # → {'count': 5, 'porcentaje': 12.2}
```

obtener_lexicon() / obtener_umbrales()

`obtener_lexicon()` retorna una copia de solo lectura del léxico (dict de 256 entradas palabra → peso). `obtener_umbrales()` retorna dict con claves `muy_positivo` y `muy_negativo`.

alertas.py — Gestión de alertas

GestorAlertas

```
GestorAlertas(umbral=-2.0, porcentaje_crisis=30.0)
```

Parámetro	Tipo	Default	Descripción
umbral	float	-2.0	Puntuación por debajo de la cual se genera alerta individual
porcentaje_crisis	float	30.0	% mínimo de comentarios negativos para alerta global

verificar_comentario(comentario)

Retorna: dict (alerta) si `puntuacion < umbral`, `None` en caso contrario.

Estructura de la alerta retornada:

Clave	Tipo	Descripción
id_comentario	str	ID del comentario que disparó la alerta
texto	str	Texto original del comentario
usuario	str	Autor del comentario (vacío si no se conoce)
puntuacion	float	Puntuación que disparó la alerta
umbral	float	Umbral configurado en este gestor
fecha_alerta	str	Timestamp YYYY-MM-DD HH:MM:SS
tipo	str	"individual"
activa	bool	True

verificar_crisis_global(estadisticas, total_comentarios)

Retorna: dict (alerta global) con `tipo = "crisis_global"` e `id_comentario = "GLOBAL"`, o `None` si el porcentaje no supera el umbral. La puntuación en la alerta global es el porcentaje de negativos (ej: `36.6` significa 36.6%). El texto tiene el formato:

```
Crisis detectada: 36.6% de comentarios negativos (15/41)
```

procesar_lote(datos_analizados)

Evalúa todos los comentarios del lote y genera alertas individuales. Imprime en consola cada alerta detectada con formato visual (incluye el usuario cuando está disponible). **Retorna:** `list(dict)`.

obtener_alertas() / total_alertas() / obtener_umbral() / configurar_umbral(nuevo_umbral)

Consultan el historial de alertas de la sesión y el umbral individual del gestor.

persistencia.py — Almacenamiento multi-formato

GestorPersistencia

```
GestorPersistencia(directorio_salida="datos_salida", db_path="crisis.db")
```

El constructor crea el directorio de salida si no existe e inicializa las tablas de la base de datos (usando `schema.sql` si está disponible, o el schema embebido). **Excepciones:** `RuntimeError` si SQLite no puede inicializarse.

guardar_txt(datos, nombre_archivo="comentarios.txt")

Exporta todos los comentarios en texto plano (incluye usuario, fecha, textos, categoría, puntuación y palabras clave). **Retorna:** ruta del archivo generado.

guardar_csv_filtrado(datos, categorias_filtro=None, nombre_archivo="negativos.csv")

Exporta en CSV solo los comentarios de las categorías indicadas (default `["negativo", "muy negativo"]`).

Columnas exportadas: `id`, `usuario`, `fecha`, `texto`, `texto_limpio`, `categoria`, `puntuacion`.

guardar_pickle(datos, ...) / cargar_pickle(...)

Serializa/deserializa cualquier objeto Python en un archivo Pickle binario. `cargar_pickle` lanza `FileNotFoundError` si el archivo no existe.

Operaciones DML

Método	Verbo DML	Retorna
<code>insertar_comentario(comentario)</code>	INSERT individual en <code>comentarios</code>	int — rowid generado
<code>insertar_lote_comentarios(datos)</code>	INSERT masivo en una transacción	int — filas insertadas
<code>insertar_alerta(alerta)</code>	INSERT en <code>alertas</code>	int — rowid generado
<code>seleccionar_comentarios(categoria=None, limite=None)</code>	SELECT con filtros, ordenado por fecha_proceso DESC	list[dict]
<code>seleccionar_alertas(solo_activas=False)</code>	SELECT de alertas (<code>activa = 1</code> si <code>solo_activas</code>)	list[dict]
<code>actualizar_categoria(id_original, nueva_categoria)</code>	UPDATE de categoría por ID original	int — filas afectadas
<code>resolver_alerta(id_alerta)</code>	UPDATE que marca <code>activa = 0</code>	int — filas actualizadas
<code>eliminar_comentario(id_original)</code>	DELETE por ID original	int — filas eliminadas
<code>limpiar_alertas_resueltas()</code>	DELETE de alertas con <code>activa = 0</code>	int — filas eliminadas

guardar_lote_completo(datos, alertas) → dict

Persiste datos y alertas en todos los formatos en una sola llamada. **Retorna:** dict con claves `txt`, `csv`, `pickle` (rutas), `sqlite_comentarios` y `sqlite_alertas` (conteos).

Las inserciones de alertas individuales fallidas se silencian; si una alerta no puede insertarse, el contador `sqlite_alertas` la excluye.

reportes.py — Generación de reportes

GeneradorReportes(directorio_salida="datos_salida")

Crea el directorio de salida si no existe.

generar_reporte_crisis(datos_analizados, alertas, nombre_archivo="reporte_crisis.txt")

Secciones del reporte:

- 1. Encabezado: métricas globales (total, alertas, índice de crisis, nivel)
- 2. Distribución de categorías con barras ASCII
- 3. Top 5 palabras negativas más frecuentes
- 4. Resumen de alertas (máx. 10 en detalle)
- 5. Top 5 comentarios con menor puntuación
- 6. Usuarios con más comentarios negativos (top 5; requiere la columna `usuario`)

Niveles de crisis calculados:

% de negativos	Nivel
≥ 60%	CRITICO
≥ 30%	ALTO
≥ 10%	MODERADO
< 10%	NORMAL

Retorna: str — ruta absoluta del reporte generado.

generar_resumen_consola(datos_analizados, alertas)

Imprime en consola un resumen estadístico breve. No retorna nada.

convertir_kaggle.py — Utilidad de conversión del dataset de Kaggle

Script independiente (no forma parte del pipeline) que convierte el `train.csv` de Kaggle ('NLP with Disaster Tweets', columnas `id`, `keyword`, `location`, `text`, `target`) al formato del sistema.

convertir(ruta_entrada, ruta_salida)

Valida que el CSV de entrada tenga las columnas `id` y `text` ; genera un CSV con columnas `id`, `texto`, `fecha`, `usuario` (fecha fija de demostración `2015-08-06` , usuario vacío porque el dataset no incluye autor). **Retorna:** int — filas convertidas. **Excepciones:** `FileNotFoundError` , `ValueError` .

```
python convertir_kaggle.py
# → Archivo generado: kaggle_convertido.csv (7 613 filas convertidas)
```

Esquema de la base de datos

Archivo: `schema.sql` (también embebido en `GestorPersistencia`). La base de datos `crisis.db` está compuesta por dos tablas y cuatro índices.

Tabla comentarios

Columna	Tipo	Restricción	Descripción
<code>id</code>	INTEGER	PK, AUTOINCREMENT	Clave primaria interna
<code>id_original</code>	TEXT	NOT NULL	ID del CSV original
<code>texto_original</code>	TEXT	NOT NULL	Texto sin procesar
<code>texto_limpio</code>	TEXT	NOT NULL	Texto normalizado
<code>usuario</code>	TEXT	NOT NULL, DEFAULT ''	Autor del comentario (vacío si el CSV no lo trae)
<code>fecha</code>	TEXT	NOT NULL	Fecha del comentario (del CSV)
<code>categoria</code>	TEXT	NOT NULL	Una de las cinco categorías de sentimiento
<code>puntuacion</code>	REAL	NOT NULL	Puntuación normalizada
<code>fecha_proceso</code>	TEXT	NOT NULL	Timestamp de inserción YYYY-MM-DD HH:MM:SS

Índices: `idx_comentarios_categoria` (categoria), `idx_comentarios_fecha` (fecha_proceso).

Tabla alertas

Columna	Tipo	Restricción	Descripción
<code>id</code>	INTEGER	PK, AUTOINCREMENT	Clave primaria
<code>id_comentario</code>	TEXT	NOT NULL	ID del comentario origen, o "GLOBAL"
<code>texto</code>	TEXT	NOT NULL	Texto resumido de la alerta
<code>usuario</code>	TEXT	NOT NULL, DEFAULT ''	Autor del comentario origen
<code>puntuacion</code>	REAL	NOT NULL	Puntuación o % que disparó la alerta
<code>umbral</code>	REAL	NOT NULL	Umbral configurado en el momento de la alerta
<code>fecha_alerta</code>	TEXT	NOT NULL	Timestamp YYYY-MM-DD HH:MM:SS
<code>activa</code>	INTEGER	NOT NULL, DEFAULT 1	1 = activa, 0 = resuelta

Índices: `idx_alertas_activa` (activa), `idx_alertas_comentario` (id_comentario).

Opciones del menú principal

Opción	Función	Datos requeridos
1	Cargar y analizar CSV	Ninguno
2	Ver estadísticas del análisis actual	CSV cargado en sesión
3	Consultar base de datos	Base de datos existente
4	Gestionar alertas	Base de datos existente
5	Generar reporte de crisis	CSV cargado en sesión
6	Exportar datos	CSV cargado en sesión
7	Demo DML (INSERT/SELECT/UPDATE/DELETE)	CSV cargado en sesión

8	Configurar umbral de alertas	Ninguno
9	Crear CSV de ejemplo	Ninguno
0	Salir	Ninguno

Véase también

- **Tutorial de inicio** — primer análisis paso a paso
- **Guías de tareas comunes** — cómo cargar CSV, exportar, consultar BD
- **Arquitectura del sistema** — diseño y decisiones de implementación